

Witbank Elite Car Wash

INT316D — Web Development Group Project

Semester 1, 2026

Individual Reflection Statements

Student Name: M.M MATHEBULA

Student ID: 221705815

Role: Database Manager

Introduction

As the Database Manager for the Witbank Elite Car Wash project, my semester was defined by a question that is central to every information system: how do we store, organise, and retrieve data in a way that is reliable, efficient, and meaningful? While our initial implementation used in-memory Java collections — a pragmatic choice given the project timeline — my responsibility extended well beyond that: I designed the complete relational database schema, the data dictionary, the seed data, and the migration path from in-memory to persistent storage. This reflection documents the database work I performed, my collaboration with the development team, the challenges I encountered, and the data management principles I have internalised through this experience.

Technical Contribution and Implementation

My first task was to perform a thorough analysis of the application's domain model by reading the Java model classes — Booking, ServicePackage, InventoryItem, and NotificationLog — and the BookingService business logic. From this analysis, I derived the four core entities of our relational schema and mapped every Java field to an appropriate SQL column type, constraint, and index.

The `service_packages` table stores the three car wash tiers (Express at R150, Elite at R450, and Platinum at R850) with their descriptions, pricing, and emoji icons. I added an `is_active` flag to support future soft-deletion of packages without breaking historical booking records — a referential integrity decision that the team initially overlooked. The `bookings` table uses a BIGINT auto-incremented primary key and a foreign key to `service_packages`, ensuring that no booking can reference a non-existent service. I enforced the booking status values ('Pending', 'Confirmed', 'In Progress', 'Completed', 'Cancelled') using a CHECK constraint, which prevents invalid state transitions at the database level rather than relying solely on application-layer validation.

The `inventory_items` table introduced a `threshold` column that does not exist in the Java model — it captures the low-stock alert level derived from the `InventoryItem.isLowStock()` business method, making that logic expressible directly in SQL. This meant I could create a `vw_low_stock_items` view that any reporting tool can query without needing to invoke Java business logic. I applied the same pattern to create views for revenue analysis

(vw_revenue_by_service), customer visit counts and VIP tier classification (vw_customer_visits), and daily booking summaries (vw_daily_bookings) — all mirroring the calculations performed in BookingService.

I also wrote two stored procedures: sp_update_booking_status, which updates a booking's status and automatically inserts the corresponding SMS and email notification log entries on completion, and sp_deduct_inventory_on_booking, which deducts consumed inventory quantities. These procedures consolidate multi-step operations into atomic database transactions, ensuring data consistency even if an application-layer error interrupts a workflow mid-execution.

Collaboration and Group Dynamics

My collaboration with the Backend Developer was the most technically intensive relationship in the team. I had to study the BookingService Java code in detail to ensure my schema mirrored its data model precisely — column names, data types, and foreign key relationships had to align exactly with what the application expected. We held several synchronisation sessions where I walked through the SQL schema against the Java code to verify consistency.

I also worked with the Tester to generate representative seed data that exercised every meaningful code path: a customer with three bookings to trigger VIP status, an inventory item below its threshold to trigger the low-stock alert, bookings in various status states to populate the admin dashboard, and notification log entries for completed bookings. This seed data transformed the test environment from an empty database into a realistic operational snapshot.

One tension I navigated was the project's initial decision to defer database integration in favour of in-memory storage. While this was the right pragmatic choice for the first sprint, it meant my schema work — though complete — was not exercised by the running application during the presentation period. I documented a clear integration guide (the application.properties changes required and the Spring Data JPA dependencies to add) to enable a future sprint to complete the persistent storage migration with minimal friction.

Lessons Learned and Future Outlook

The most important lesson I learned is that a database schema is not just a storage mechanism — it is a formalisation of the business's understanding of its own domain. Every constraint, foreign key, and index I wrote encodes a business rule. The CHECK constraint on booking status says: these are the only valid states a booking can be in. The foreign key between bookings and service_packages says: you cannot book a service that does not exist. Getting these rules right at the database level provides a safety net that protects data integrity regardless of application bugs.

I also developed a deep appreciation for the difference between OLTP (transactional) and OLAP (analytical) data patterns. The views I created — aggregating revenue, counting visits, flagging low stock — are early steps toward an analytical layer on top of a transactional system. As data volumes grow, these would need to be materialised or moved to a separate reporting database. Understanding this architectural distinction earlier would have strengthened my initial schema decisions.

Looking forward, I intend to study Spring Data JPA to bridge the gap between the Java application and the SQL database I designed, and to explore database performance

optimisation — indexing strategies, query plans, and normalisation trade-offs — as my next area of growth.

Conclusion

Designing the database for Witbank Elite Car Wash was an exercise in translating a business's operational reality into a precise, durable, machine-readable structure. Every table, constraint, view, and stored procedure represents a considered decision about what data matters, how it relates, and how it should be protected. I am proud of the schema I delivered and grateful for the collaboration that challenged and sharpened my thinking. This project has confirmed my passion for data engineering, and I look forward to building systems where the database layer is treated as a first-class architectural concern.